

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



Đặng Ngọc Kim Khánh - 52300208
Vũ Khánh Huyền - 52300206

REACT-BOOTSTRAP

BÁO CÁO GIỮA KỲ
PHÁT TRIỂN PHẦN MỀM
FULL-STACK

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN



Đặng Ngọc Kim Khánh - 52300208
Vũ Khánh Huyền - 52300206

REACT-BOOTSTRAP

BÁO CÁO GIỮA KỲ
PHÁT TRIỂN PHẦN MỀM
FULL-STACK

Người hướng dẫn
ThS.NCS.Vũ Đình Hồng

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Chúng em xin chân thành bày tỏ sự biết ơn sâu sắc và lòng kính trọng đến ThS.NCS.Vũ Đình Hồng, thầy là người hướng dẫn chúng em trong dự án lần này. Trong quá trình học tập và làm việc, thầy đã truyền đạt cho chúng em vô vàn kiến thức hay và bổ ích, giúp chúng em có được cơ sở lý thuyết vững vàng để em có thể hoàn thành dự án này.

Tuy nhiên, trong quá trình thực hiện đồ án, vì sự hiểu biết còn hạn chế của chúng em, bài báo cáo còn nhiều sai sót và chưa chính chu như chúng em mong muốn. Chúng em rất mong nhận được nhận xét và đánh giá của thầy cô để có thể rút kinh nghiệm cũng như sửa chữa lỗi sai của bản thân.

Chúng em xin kính chúc quý thầy, quý cô và quý nhà trường luôn mạnh khỏe, hạnh phúc và ngày một thành công hơn trong sự nghiệp trồng người của mình.

TP. Hồ Chí Minh, ngày 27 tháng 04 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

Đặng Ngọc Kim Khánh

Vũ Khánh Huyền

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Tôi xin cam đoan đây là công trình nghiên cứu của riêng tôi và được sự hướng dẫn khoa học của ThS.NCS.Vũ Đình Hồng. Các nội dung nghiên cứu, kết quả trong đề tài này là trung thực và chưa công bố dưới bất kỳ hình thức nào trước đây. Những số liệu trong các bảng biểu phục vụ cho việc phân tích, nhận xét, đánh giá được chính tác giả thu thập từ các nguồn khác nhau có ghi rõ trong phần tài liệu tham khảo.

Ngoài ra, trong Dự án còn sử dụng một số nhận xét, đánh giá cũng như số liệu của các tác giả khác, cơ quan tổ chức khác đều có trích dẫn và chú thích nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào tôi xin hoàn toàn chịu trách nhiệm về nội dung Dự án của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày 27 tháng 04 năm 2026

Tác giả

(Ký tên và ghi rõ họ tên)

Đặng Ngọc Kim Khánh

Vũ Khánh Huyền

TÓM TẮT

Báo cáo giữa kỳ này trình bày quá trình tìm hiểu và ứng dụng thư viện React-Bootstrap trong phát triển giao diện người dùng cho ứng dụng web hiện đại. React-Bootstrap là một thư viện UI mã nguồn mở, kết hợp sức mạnh của React và Bootstrap, cho phép xây dựng các giao diện responsive một cách nhanh chóng thông qua hệ thống component có sẵn.

Trong báo cáo, nhóm tập trung tìm hiểu các component cốt lõi của thư viện bao gồm **Navbar**, **Nav**, **Table**, **Accordion**, **Form**, **Alert** và **Modal**, đồng thời phân tích cách tích hợp chúng vào ứng dụng React thực tế. Để minh họa, nhóm xây dựng một **mini-app quản lý sản phẩm** hỗ trợ đầy đủ các chức năng thêm, sửa, xóa và hiển thị danh sách sản phẩm, trong đó toàn bộ giao diện được xây dựng bằng các component của React-Bootstrap.

Kết quả cho thấy React-Bootstrap giúp rút ngắn đáng kể thời gian phát triển giao diện, đảm bảo tính nhất quán về mặt thiết kế và dễ dàng tích hợp với hệ sinh thái React. Đây là lựa chọn phù hợp cho các dự án cần triển khai nhanh với giao diện chuyên nghiệp.

MỤC LỤC

DANH MỤC HÌNH VẼ.....	7
DANH MỤC BẢNG BIỂU.....	8
DANH MỤC CÁC CHỮ VIẾT TẮT.....	9
CHƯƠNG 1. GIỚI THIỆU REACT-BOOTSTRAP.....	1
1.1 React-bootstrap là gì?.....	1
1.2 Mục đích sử dụng.....	1
1.3 Cách cài đặt.....	1
1.4 Ưu và nhược điểm.....	2
CHƯƠNG 2. CÁC COMPONENT CHÍNH.....	3
2.1 Navigation: Navbar, Nav, NavDropdown.....	3
2.2 Table, Card và Accordion.....	4
2.2.1 Table - Bảng dữ liệu.....	4
2.2.2 Accordion - Thu gọn nội dung.....	5
2.2.3 Card - Khung thông tin tổng quan.....	6
2.3 Form.....	7
2.4 Alert.....	8
2.5 Modal.....	9
CHƯƠNG 3. MINI-APP MINH HOẠ.....	11
3.1 Giới thiệu mini-app quản lý sản phẩm.....	11
3.1.1 Mục tiêu và phạm vi.....	11
3.1.2 Các chức năng chính.....	11
3.2 Cấu trúc project.....	12
3.2.1 Cây thư mục.....	12
3.2.2 Vai trò từng file quan trọng.....	13
3.3 Giao diện chính và kết quả minh hoạ.....	14

3.3.1 Thanh điều hướng (Navbar) và trang Home.....	14
3.3.2 Trang danh sách sản phẩm.....	16
3.3.3 Trang thêm sản phẩm mới.....	18
3.3.4 Giao diện responsive.....	20
CHƯƠNG 4. BÀI TẬP ỨNG DỤNG.....	22
4.1 Bài 1: Navbar.....	22
4.1.1 Yêu cầu.....	22
4.1.2 Phân tích kỹ thuật.....	22
4.1.3 Code minh hoạ.....	22
4.1.4 Giải thích chi tiết.....	24
4.2 Bài 2: Table + Modal.....	24
4.2.1 Yêu cầu.....	24
4.2.2 Phân tích kỹ thuật.....	24
4.2.3 Code minh hoạ.....	25
4.2.4 Giải thích chi tiết.....	28
4.3 Bài 3: Form + Alert.....	28
4.3.1 Yêu cầu.....	28
4.3.2 Phân tích kỹ thuật.....	28
4.3.3 Code minh hoạ.....	29
4.3.4 Giải thích chi tiết.....	29
CHƯƠNG 5. HƯỚNG DẪN CHẠY PROJECT VÀ KẾT LUẬN.....	31
5.1 Hướng dẫn chạy.....	31
5.1.1 Yêu cầu môi trường.....	31
5.1.2 Các bước thực hiện.....	31
5.1.3 Kiểm tra cài đặt thành công.....	32
5.2 Kết luận.....	32

TÀI LIỆU THAM KHẢO.....	33
--------------------------------	-----------

DANH MỤC HÌNH VẼ

Hình 3.1: Sơ đồ cấu trúc project.....	13
Hình 3.2: Giao diện thanh điều hướng và trang Home đang được chọn.....	15
Hình 3.4: Giao diện trang “Danh sách” chứa 10 bản ghi dữ liệu.....	17
Hình 3.5: Giao diện Modal sửa sản phẩm.....	17
Hình 3.6: Giao diện Modal xác nhận xóa sản phẩm.....	18
Hình 3.7: Giao diện thông báo nhập form thành công.....	19
Hình 3.8: Giao diện form hiển thị lỗi khi chưa điền đầy đủ thông tin.....	19
Hình 3.9: Giao diện responsive của trang Home.....	20
Hình 3.10: Giao diện của bảng dữ liệu có scroll ngang khi ở responsive.....	21

DANH MỤC BẢNG BIỂU

Bảng 1.1: Bảng so sánh ưu và nhược điểm.....	2
Bảng 2.1: Bảng các thuộc tính quan trọng.....	4
Bảng 2.2: Bảng thuộc tính Table.....	5
Bảng 2.3: Bảng mô tả thuộc tính của Accordion.....	6
Bảng 2.4: Bảng thuộc tính quan trọng của Form.....	8
Bảng 2.5: Bảng thuộc tính quan trọng của Alert.....	9
Bảng 2.6: Bảng thuộc tính quan trọng của Model.....	10
Bảng 3.1: Bảng tổng hợp chức năng và component tương ứng.....	12
Bảng 3.2: Bảng vai trò các tệp trong project.....	14
Bảng 5.1: Bảng môi trường cần thiết để chạy project.....	31

DANH MỤC CÁC CHỮ VIẾT TẮT

DOM	Document Object Model
SPA	Single Page Application

CHƯƠNG 1. GIỚI THIỆU REACT-BOOTSTRAP

1.1 React-bootstrap là gì?

React-Bootstrap là một thư viện giao diện người dùng mã nguồn mở, được phát triển nhằm tích hợp Bootstrap vào hệ sinh thái React. Thay vì thao tác trực tiếp với DOM thông qua jQuery như Bootstrap truyền thống, React-Bootstrap cung cấp các component React hoàn chỉnh, cho phép lập trình viên xây dựng giao diện thông qua JSX và quản lý trạng thái bằng React state.

1.2 Mục đích sử dụng

- Xây dựng giao diện web responsive nhanh chóng và không cần tự viết CSS từ đầu
- Tái sử dụng component linh hoạt trong nhiều màn hình khác nhau
- Tích hợp tốt với React state và props, giúp giao diện phản ứng theo dữ liệu
- Phù hợp với các dự án vừa và nhỏ cần triển khai nhanh

1.3 Cách cài đặt

Tạo project React

```
npm create vite@latest ten-project -- --template react
cd ten-project
npm install
```

Cài React-Bootstrap

Dùng lệnh: `npm install react-bootstrap bootstrap`

Cài thêm React-Router

```
npm install react-router-dom
```

- React-Router là thư viện định tuyến phổ biến, cho phép xây dựng ứng dụng Single Page Application (SPA) với nhiều trang khác nhau mà không cần tải lại toàn bộ trang web. Thay vì chuyển hướng theo kiểu truyền thống,

React-Router quản lý URL và render đúng component tương ứng dựa trên đường dẫn hiện tại.

- Các khái niệm cơ bản cần nhắc đến: *BrowserRouter*, *Routes/ Route*, *Link/ NavLink*, *useNavigate*

Import CSS vào index.jsx

```
import 'bootstrap/dist/css/bootstrap.min.css'
```

Sử dụng component:

Để sử dụng các component thì ta import các component cần dùng vào đầu file, ví dụ:

```
import { Container, Card, Form, Row, Col, Button, Alert }
from "react-bootstrap";
```

1.4 Ưu và nhược điểm

Ưu điểm	Nhược điểm
Dễ sử dụng, cú pháp quen thuộc với React	Bundle size lớn hơn so với dùng CSS thuần
Nhiều component có sẵn, tiết kiệm thời gian	Khó tùy chỉnh style sâu nếu không dùng thêm CSS
Responsive sẵn, không cần cấu hình thêm	Giao diện mặc định dễ trùng lặp với nhiều dự án khác
Không phụ thuộc jQuery, nhẹ hơn Bootstrap cũ	Cần học thêm prop API riêng của từng component

Bảng 1.1: Bảng so sánh ưu và nhược điểm

CHƯƠNG 2. CÁC COMPONENT CHÍNH

2.1 Navigation: Navbar, Nav, NavDropdown

Hệ thống điều hướng là thành phần đầu tiên người dùng tiếp xúc khi mở ứng dụng. Trong React-Bootstrap, thanh navigation được xây dựng từ ba component phối hợp với nhau: Navbar, Nav và NavDropdown.

- **Navbar:** Đóng vai trò là container bao ngoài toàn bộ thanh điều hướng. Component này hỗ trợ responsive thông qua prop `expand` - cho phép chỉ định breakpoint mà tại đó thanh điều hướng sẽ thu gọn thành nút menu (`expand = "lg"`: thu gọn màn hình dưới 992px)
- **Nav:** là nhóm chứa các mục điều hướng (menu items). Có thể đặt ở bất kỳ vị trí nào trong Navbar
- **Nav.Link:** tương đương thẻ `<a>` trong HTML thuần, dùng để tạo từng mục menu có thể chọn được
- **NavDropdown:** tạo menu thả xuống khi người dùng click vào mục dropdown đó thì sẽ có danh sách các lựa chọn

Tích hợp với React Router: Vấn đề cốt lõi khi kết hợp React-Bootstrap với React-Router là tránh việc tải lại toàn bộ trang mỗi khi người dùng nhấn vào một mục menu. Cách tiếp cận:

- Dùng prop `as={NavLink}`: truyền trực tiếp component `NavLink` của React Router vào prop `as` của `Nav.Link`. React-Bootstrap sẽ render component đó thay vì thẻ `<a>` mặc định. Ví dụ: `<Nav.Link as={NavLink} to="/list">`

Về active state: Khi sử dụng `NavLink`, React Router tự động gán class `.active` vào mục đang được chọn (URL khớp với href). React-Bootstrap đọc class này và highlight mục đó, giúp người dùng biết mình đang ở trang nào.

Thuộc tính	Giá trị mẫu	Mô tả
expand	“sm” “md” “lg” “xl”	Breakpoint để thu gọn nav thành nút hamburger menu
bg	“light” “dark” “primary”	Màu nền của Navbar
variant	“light” “dark”	Màu chữ/ icon trong Navbar
fixed	“top” “bottom”	Cố định Navbar trên cùng hoặc dưới cùng màn hình
as {Nav.Link}	{Nav.Link}	Thay thế thẻ render mặc định bằng component khác

Bảng 2.1: Bảng các thuộc tính quan trọng

2.2 Table, Card và Accordion

Đây là hai component đảm nhận nhiệm vụ trình bày dữ liệu theo các hình thức khác nhau: Table hiển thị danh sách dạng bảng, Accordion tổ chức nội dung theo dạng thu gọn/ mở rộng.

2.2.1 Table - Bảng dữ liệu

Component Table của React-Bootstrap là phần mở rộng trực tiếp từ thẻ <table> trong HTML, được bổ sung thêm các prop tiện ích để styling nhanh mà không cần viết CSS thủ công.

- `striped`: Tạo màu nền xen kẽ giữa các dòng, giúp người dùng đọc dữ liệu dễ hơn khi bảng có nhiều cột.

- **bordered**: Hiển thị đường viền quanh toàn bộ ô trong bảng, thích hợp khi cần phân tách dữ liệu rõ ràng.
- **hover**: Thêm hiệu ứng đổi màu khi di chuột qua một hàng, hỗ trợ trải nghiệm người dùng tốt hơn.
- **responsive**: Bọc bảng trong một div có overflow-x: auto, cho phép scroll ngang trên thiết bị di động khi bảng quá rộng.
- **condensed**: Cắt giảm bớt khoảng trắng (padding). Giúp bảng gọn hơn, hiển thị được nhiều dữ liệu hơn trên một màn hình.

Tên thuộc tính	Thẻ loại	Giá trị mẫu mặc định	Mô tả
striped	boolean	False	Màu nền xen kẽ cho các dòng
bordered	boolean	False	Hiển thị viền cho tất cả ô trong bảng
hover	boolean	False	Highlight hàng khi di chuột
responsive	boolean	False	Cho phép scroll ngang trên màn hình nhỏ
condensed	boolean	False	Cắt bớt padding

Bảng 2.2: Bảng thuộc tính Table

2.2.2 Accordion - Thu gọn nội dung

Accordion là một dạng mở rộng của Card, cho phép ẩn/ hiện nội dung khi người dùng nhấn vào tiêu đề.

React-Bootstrap cung cấp hai chế độ hoạt động của Accordion:

- Mặc định (một panel mở): Chỉ một Accordion.Item có thể mở tại một thời điểm. Khi mở item mới, item đang mở sẽ tự động đóng lại.
- `alwaysOpen`: Truyền prop `alwaysOpen` vào Accordion để cho phép nhiều item mở cùng lúc.

Thuộc tính	Mô tả
<code>defaultActiveKey</code>	Xác định mục (eventKey) sẽ được mở sẵn khi component khởi tạo lần đầu
<code>activeKey</code>	Dùng để kiểm soát trạng thái đóng/ mở của Accordion thông qua State
<code>alwaysOpen</code>	Cho phép mở nhiều mục cùng lúc thay vì chỉ được mở một mục
<code>eventKey (Required)</code>	ID duy nhất của mỗi mục. Kết nối với <code>defaultActiveKey</code> của thẻ cha. Khi click vào tiêu đề Accordion sẽ dựa vào key này để biết cần mở mục nào và đóng mục nào

Bảng 2.3: Bảng mô tả thuộc tính của Accordion

2.2.3 Card - Khung thông tin tổng quan

Component Card trong React-Bootstrap là sự thay thế cho "Panel" trong Bootstrap 3 cũ. Card được dùng để bọc các khối thông tin tổng quan như thống kê, thông báo quan trọng, hoặc các widget nhỏ. Cấu trúc điển hình gồm Card.Header, Card.Body, và Card.Footer.

```
<Card
  className="shadow-sm border-0 rounded-3 h-100 custom-card"
  onClick={handleDevelopingFeature}
  style={{ cursor: "pointer" }}
>
```

```

    >
    <Card.Body className="p-4">
      <Card.Subtitle className="mb-2 text-muted small fw-bold">
        {item.title}
      </Card.Subtitle>

      <Card.Title
        as="h1"
        className={`display-4 fw-bold mb-1 ${item.color}`}
      >
        {item.value}
      </Card.Title>
    </Card.Body>
  </Card>

```

2.3 Form

Form là component được sử dụng để xây dựng giao diện nhập liệu trong ứng dụng React. Trong React-Bootstrap, Form cung cấp sẵn nhiều thành phần hỗ trợ như ô nhập văn bản, email, password, checkbox, select box,... giúp việc xây dựng giao diện trở nên nhanh chóng và đồng bộ với Bootstrap.

Trong ứng dụng, trang **Add** (thêm sản phẩm) sử dụng hệ thống Form của React-Bootstrap để thu thập dữ liệu cho các trường: name, cat, price, và qty.

- **Cấu trúc phân cấp:** Chúng ta sử dụng `Form.Group` để bao bọc từng cặp nhãn (`Form.Label`) và ô nhập liệu (`Form.Control`). Điều này giúp duy trì khoảng cách và cấu trúc đồng nhất cho giao diện.
- **Grid Layout trong Form:** Để tối ưu không gian, các trường như "Giá" và "Số lượng" có thể được đặt trên cùng một hàng bằng cách kết hợp các component `Row` và `Col`.

- **Validation (Kiểm duyệt):** Sử dụng thuộc tính `required` trong `Form.Control` để bắt buộc người dùng nhập liệu.
 - Sử dụng `Form.Control.Feedback` với `type="invalid"` để hiển thị thông báo lỗi cụ thể (ví dụ: "Vui lòng nhập tên sản phẩm") khi người dùng nhấn nút "Lưu" mà để trống thông tin.

Component	Thuộc tính	Giá trị mẫu	Mô tả
Form	validated	{true} {false}	Kích hoạt kiểu hiển thị lỗi/thành công của Bootstrap.
Form.Control	type	"text","number"	Xác định kiểu dữ liệu đầu vào.
	placeholder	"Nhập tên SP..."	Văn bản gợi ý hiển thị khi ô trống.
Feedback	type	"invalid"	Hiển thị thông báo khi dữ liệu không hợp lệ.

Bảng 2.4: Bảng thuộc tính quan trọng của Form

2.4 Alert

Alert là component dùng để hiển thị thông báo trạng thái cho người dùng sau khi thực hiện thao tác trong hệ thống. Component này giúp người dùng dễ dàng nhận biết thao tác đã thành công, thất bại hay cần chú ý.

Trong bài làm của nhóm, component **Alert** được sử dụng để cung cấp phản hồi ngay lập tức cho người dùng sau khi họ thực hiện các thao tác thêm hoặc xóa sản phẩm.

Màu sắc theo ngữ nghĩa:

- `variant="success"`: Hiển thị khi thêm sản phẩm thành công hoặc xóa thành công.

- `variant="danger"`: Hiển thị khi form nhập liệu thiếu thông tin hoặc có lỗi hệ thống.

Quản lý hiển thị: Alert được điều khiển thông qua một biến state (ví dụ: `showMsg`). Khi người dùng nhấn nút lưu, nếu thành công, ta set state này thành `true` để hiển thị thông báo.

Tự động đóng (Auto-dismiss): Để tăng trải nghiệm, có thể dùng `setTimeout` trong Hook `useEffect` để tự động ẩn Alert sau 3 giây, giúp người dùng không phải tắt alert thủ công.

Thuộc tính	Giá trị mẫu	Mô tả
<code>variant</code>	<code>"success" "danger" "warning"</code>	Xác định màu sắc và biểu tượng theo ngữ nghĩa.
<code>dismissible</code>	<code>{true}</code>	Thêm nút "X" ở góc để người dùng có thể tự đóng thông báo.
<code>show</code>	<code>{showState}</code>	Biến boolean quyết định việc Alert có xuất hiện hay không.

Bảng 2.5: Bảng thuộc tính quan trọng của Alert

2.5 Modal

Modal là component dùng để tạo cửa sổ popup hiển thị phía trên giao diện chính. Modal thường được sử dụng trong các tình huống cần người dùng xác nhận thao tác hoặc nhập thông tin bổ sung.

Trong bài làm, vì danh sách sản phẩm trong `App.jsx` chứa các thông tin quan trọng, việc xóa một sản phẩm cần được xác nhận thông qua **Modal** để tránh thao tác nhầm.

- **Kích hoạt Modal:** Tại mỗi dòng của bảng danh sách sản phẩm (`List.jsx`), nút "Xóa" sẽ gọi hàm mở Modal và lưu lại `id` của sản phẩm cần xóa vào state.
- **Cấu trúc nội dung:**
 - `Modal.Header`: Hiển thị tiêu đề "Xác nhận xóa sản phẩm".
 - `Modal.Body`: Hiển thị câu hỏi xác nhận (ví dụ: "Bạn có chắc chắn muốn xóa sản phẩm 'Airpods 4' không?").
 - `Modal.Footer`: Chứa nút "Hủy" (để đóng Modal) và nút "Xóa" (để thực thi hàm `setProducts` lọc bỏ ID tương ứng).
- **Tương tác:** Sử dụng prop `onHide` để đảm bảo khi người dùng nhấn phím Esc hoặc click ra ngoài vùng Modal, hộp thoại sẽ tự đóng một cách an toàn.

Thuộc tính	Giá trị mẫu	Mô tả
<code>show</code>	<code>{showModal}</code>	State quản lý việc hiển thị/ẩn của hộp thoại.
<code>onHide</code>	<code>{handleClose}</code>	Hàm xử lý khi người dùng yêu cầu đóng Modal.
<code>centered</code>	<code>{true}</code>	Căn giữa Modal theo chiều dọc màn hình thay vì sát mép trên.
<code>backdrop</code>	<code>"static"</code>	Ngăn người dùng đóng Modal khi nhấn ra vùng ngoài (buộc phải nhấn nút).

Bảng 2.6: Bảng thuộc tính quan trọng của Modal

CHƯƠNG 3. MINI-APP MINH HOẠ

3.1 Giới thiệu mini-app quản lý sản phẩm

3.1.1 Mục tiêu và phạm vi

Mini-app được thiết kế với vai trò là một hệ thống quản lý sản phẩm đơn giản, phù hợp với quy mô học thuật nhưng phản ánh đúng luồng vận hành của một ứng dụng thực tế. Người dùng có thể thực hiện đầy đủ các chức năng: xem danh sách, thêm mới, chỉnh sửa thông tin, và xóa sản phẩm.

Giới hạn phạm vi của ứng dụng: dữ liệu được lưu trong React state (useState), tức là chỉ tồn tại trong phiên làm việc hiện tại trên trình duyệt và sẽ mất đi khi tải lại trang. Đây là lựa chọn phù hợp để tập trung vào việc minh họa component React - Bootstrap mà không bị phân tán vào phần backend hay quản lý database

3.1.2 Các chức năng chính

Chức năng	Component sử dụng	Mô tả
Xem danh sách sản phẩm	<Table>	Hiển thị toàn bộ sản phẩm dạng bảng
Thêm sản phẩm	<Form>, <Button>	Form giúp người dùng nhập thông sản phẩm, và có validate
Xóa sản phẩm	<Modal>, <Button>	Modal hiển thị để xác nhận có xóa sản phẩm không; Button dùng để tạo nút Xóa hoặc Hủy
Sửa sản phẩm	<Modal>, <Button>, <Form>	Modal chứa form để người dùng có thể sửa

		sản phẩm khi bấm chọn vào nút sửa
Hiển thị thông báo xóa/ sửa sản phẩm thành công	<Alert>	Khi sửa/ xóa thành công/ không thành công Alert sẽ hiện ra thông báo
Điều hướng	<Navbar>, <Nav>, <NavLink>	Navbar responsive, highlight mục đang active tự động

Bảng 3.1: Bảng tổng hợp chức năng và component tương ứng

3.2 Cấu trúc project

3.2.1 Cây thư mục

Project được tạo bằng Vite theo template React mặc định, sau đó nhóm bổ sung thêm thư mục con bên trong src/ để tổ chức code rõ ràng:

```
[dangngockimkhanh@kimkhanhne GKF23_Full-stack % tree -I "node_modules"
```

```
.
├── README.md
├── mini-app-react
│   ├── index.html
│   ├── package-lock.json
│   ├── package.json
│   ├── public
│   │   ├── favicon.ico
│   │   ├── index.html
│   │   ├── logo192.png
│   │   ├── logo512.png
│   │   ├── manifest.json
│   │   └── robots.txt
│   └── src
│       ├── App.css
│       ├── App.jsx
│       ├── App.test.jsx
│       ├── components
│       │   ├── Add.jsx
│       │   ├── AppAlert.jsx
│       │   ├── ConfirmDeleteModal.jsx
│       │   ├── EditProductModal.jsx
│       │   ├── Home.jsx
│       │   ├── List.jsx
│       │   ├── MyNavBar.jsx
│       │   └── ProductForm.jsx
│       ├── index.css
│       ├── index.jsx
│       ├── logo.svg
│       ├── reportWebVitals.jsx
│       ├── setupTests.jsx
│       └── vite.config.js
└── 5 directories, 27 files
```

Hình 3.1: Sơ đồ cấu trúc project

3.2.2 Vai trò từng file quan trọng

Tên file	Chức năng
App.jsx	Quản lý trạng thái tổng, định tuyến đường dẫn và bao bọc toàn bộ ứng dụng
Add.jsx	Trang thêm mới sản phẩm: cung cấp

	giao diện cho người dùng nhập sản phẩm mới vào danh sách
AppAlert.jsx	Component thông báo: Hiển thị thông báo phản hồi
ConfirmDeleteModal.jsx	Hộp thoại xác nhận xoá: Hiển thị Modal xác nhận xoá sản phẩm
EditProductModal.jsx	Hộp thoại chỉnh sửa: Hiển thị modal chỉnh sửa sản phẩm khi bấm chọn nút Sửa
Home.jsx	Trang chủ: Hiển thị các Card thông tin tổng quan và Accordion
List.jsx	Trang danh sách: Hiển thị danh sách sản phẩm dạng bảng, chứa logic xử lý chính như xem, sửa, xoá sản phẩm
MyNavBar.jsx	Thanh điều hướng: Chứa các liên kết di chuyển đến các trang và hiển thị thông báo “Phần mềm đang phát triển”
ProductForm.jsx	Tạo form tại trang thêm sản phẩm mới, form chứa các ô nhập liệu và có validate

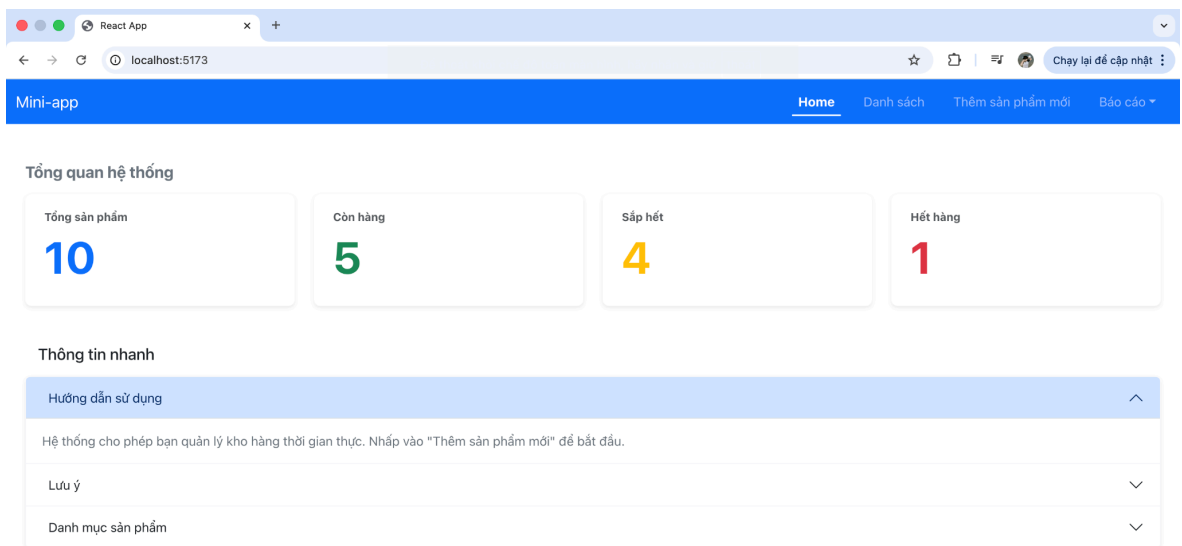
Bảng 3.2: Bảng vai trò các tệp trong project

3.3 Giao diện chính và kết quả minh hoạ

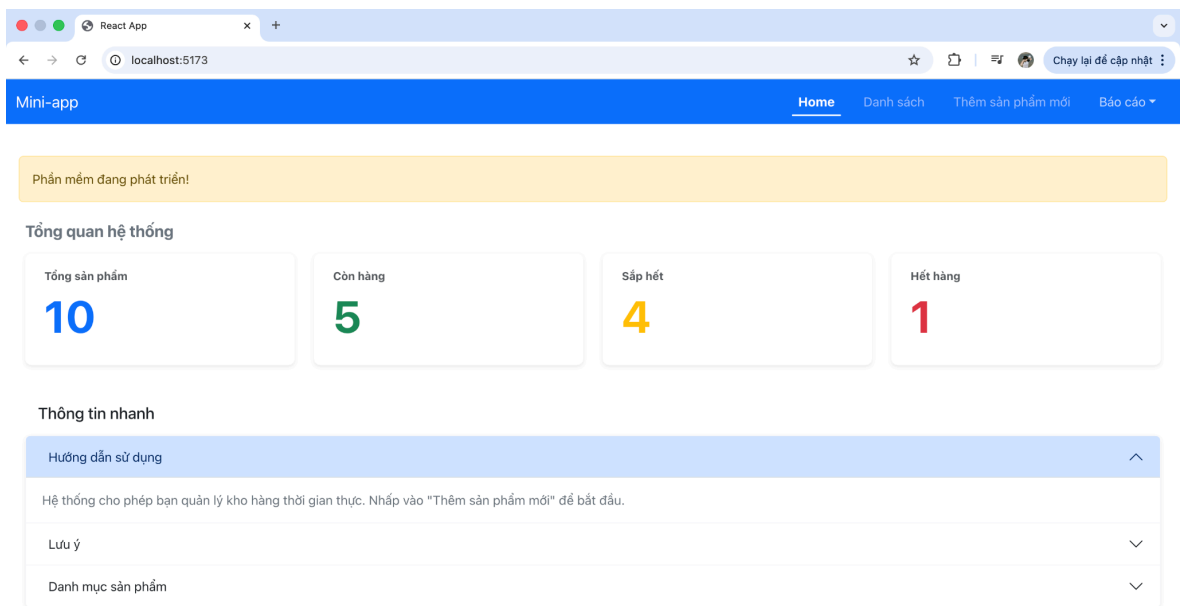
3.3.1 Thanh điều hướng (Navbar) và trang Home

Navbar được cố định ở trên cùng và hiển thị nhất quán trên tất cả các trang. Ba mục điều hướng Home, Danh sách, Thêm sản phẩm mới và NavDropdown Báo cáo đều dùng NavLink nên mục đang active sẽ được tự động được highlight bằng màu trắng đậm hơn so với các mục còn lại và có dấu gạch chân ở dưới.

Trang Home là trang chính, chứa các nội dung tổng quan, bao gồm các component Card và Accordion. “Báo cáo” thuộc component NavDropdown, khi xổ xuống sẽ có 3 chức năng, tuy nhiên khi bấm chọn một trong ba sẽ hiển thị thông báo “Phần mềm đang phát triển”.



Hình 3.2: Giao diện thanh điều hướng và trang Home đang được chọn



Hình 3.3: Giao diện khi hiển thị thông báo đang phát triển cho mục “Báo cáo”

3.3.2 Trang danh sách sản phẩm

Đây là trang hiển thị danh sách các sản phẩm mà ứng dụng quản lý, được hiển thị dưới dạng bảng, bên cạnh đó, người dùng có thể thao tác sửa xóa thông qua các Button. Bảng dữ liệu dùng Table với các prop hover,... để tạo giao diện dễ nhìn hơn. Cột “Trạng thái” sử dụng Badge để tạo giao diện các nhãn hiển thị trạng thái của hàng (“Còn hàng”, “Sắp hết”, “Hết hàng”).

Khi bấm vào nút Sửa, Modal chứa form cho phép người dùng chỉnh sửa thông tin của sản phẩm; khi bấm vào nút Xóa, Modal hiển thị để xác nhận có muốn xóa sản phẩm hay không.

Nút “Thêm mới” ở góc trên bên phải của bảng dữ liệu, khi bấm vào thì sẽ được chuyển sang trang “Thêm sản phẩm mới”

React App
localhost:5173/list

Mini-app Home **Danh sách** Thêm sản phẩm mới Báo cáo

Danh sách sản phẩm Thêm mới

STT	TÊN SẢN PHẨM	DANH MỤC	ĐƠN GIÁ	KHO	TRẠNG THÁI	THAO TÁC
1	Airpods 4	Điện tử	6.500.000	0	Hết hàng	Sửa Xoá
2	Đầm công chúa	Thời trang	500.000	5	Sắp hết	Sửa Xoá
3	iPhone 15	Điện tử	18.990.000	8	Còn hàng	Sửa Xoá
4	Samsung Galaxy S24	Điện tử	20.990.000	10	Còn hàng	Sửa Xoá
5	Áo hoodie unisex	Thời trang	350.000	25	Còn hàng	Sửa Xoá
6	Quần jean nam	Thời trang	420.000	18	Còn hàng	Sửa Xoá
7	Sữa tươi Vinamilk 1L	Thực phẩm	32.000	50	Còn hàng	Sửa Xoá
8	Mì gói Hào Hào	Thực phẩm	4.000	200	Còn hàng	Sửa Xoá
9	Nồi cơm điện Sharp	Gia dụng	890.000	12	Còn hàng	Sửa Xoá
10	Máy xay sinh tố	Gia dụng	650.000	7	Còn hàng	Sửa Xoá

Hình 3.4: Giao diện trang “Danh sách” chứa 10 bản ghi dữ liệu

Cập nhật sản phẩm ×

Tên sản phẩm Danh mục

Airpods 4 Điện tử

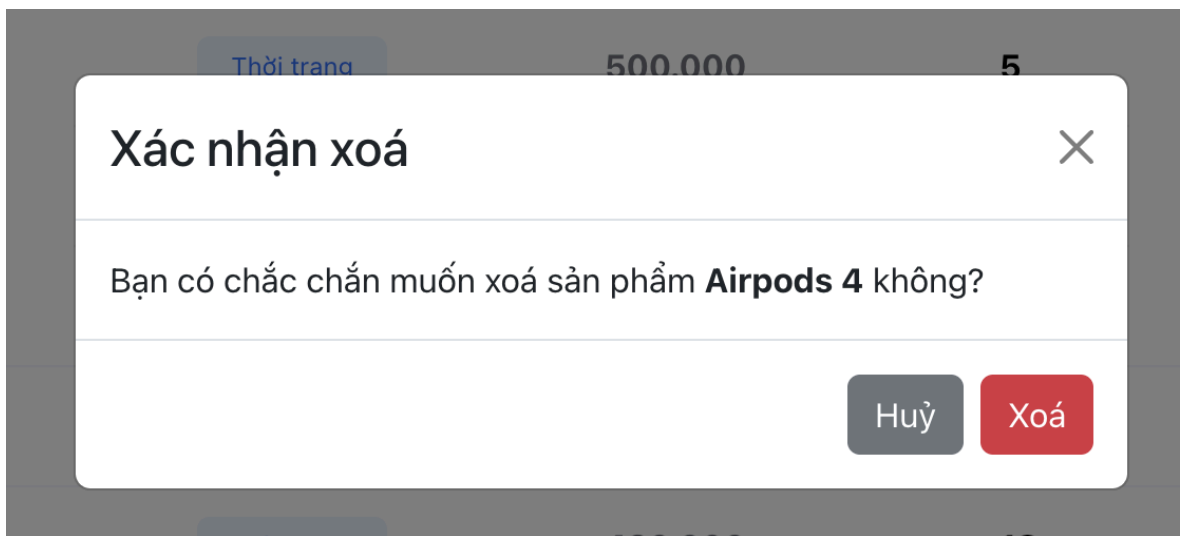
Đơn giá Số lượng

6.500.000 0

Huỷ Lưu thay đổi

Thực phẩm 32.000 50 Còn hàng

Hình 3.5: Giao diện Modal sửa sản phẩm

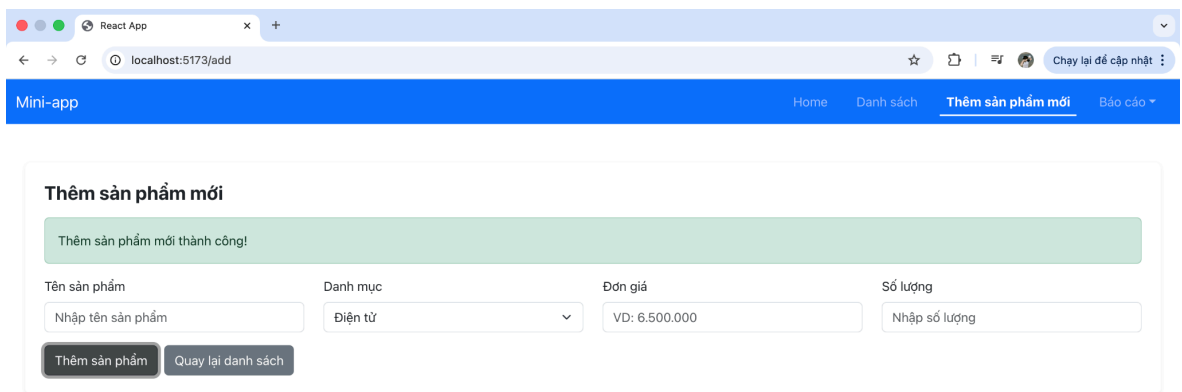


Hình 3.6: Giao diện Modal xác nhận xoá sản phẩm

3.3.3 Trang thêm sản phẩm mới

Form thêm sản phẩm gồm 4 trường: Tên sản phẩm, Giá, Danh mục và Số lượng. Khi người dùng nhấn Lưu mà chưa điền đầy đủ, hệ thống hiển thị Alert màu đỏ (danger); khi dữ liệu hợp lệ thì hiển thị Alert xanh (success) và đồng thời tự reset form về trạng thái trống.

Tại trường “Đơn giá”, chỉ chấp nhận dữ liệu số, nếu nhập chữ, thì hệ thống không cho phép nhập.



The screenshot shows a web browser window with the address bar at localhost:5173/add. The application has a blue navigation bar with links: Mini-app, Home, Danh sách, Thêm sản phẩm mới (active), and Báo cáo. The main content area is titled "Thêm sản phẩm mới" and displays a green success message: "Thêm sản phẩm mới thành công!". Below the message is a form with four fields: "Tên sản phẩm" (with placeholder "Nhập tên sản phẩm"), "Danh mục" (a dropdown menu showing "Điện tử"), "Đơn giá" (with placeholder "VD: 6.500.000"), and "Số lượng" (with placeholder "Nhập số lượng"). At the bottom of the form are two buttons: "Thêm sản phẩm" and "Quay lại danh sách".

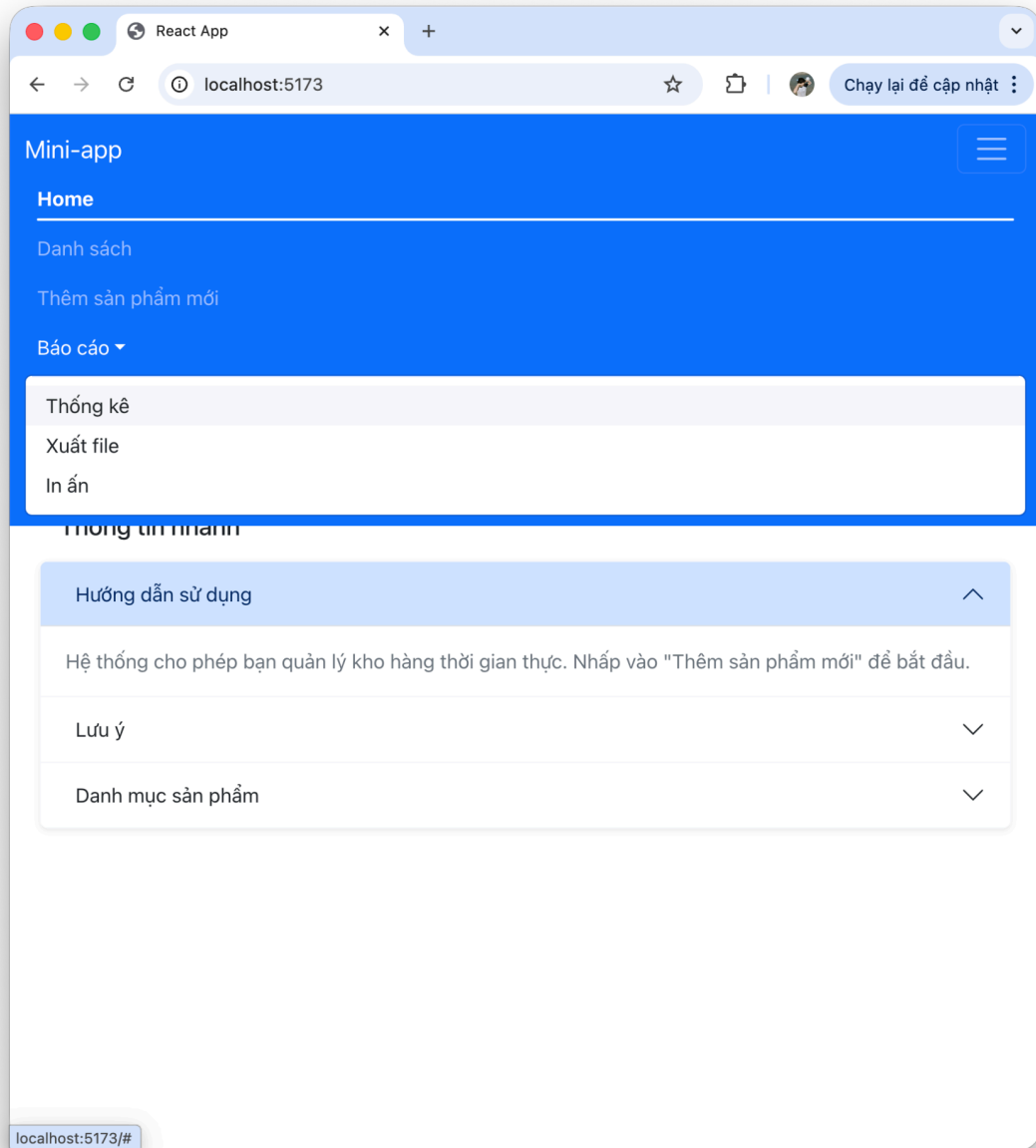
Hình 3.7: Giao diện thông báo nhập form thành công



The screenshot shows the same web application as in Figure 3.7, but with a red error message: "Vui lòng nhập đầy đủ tên sản phẩm, giá và số lượng!". The form fields are the same: "Tên sản phẩm" (placeholder "Nhập tên sản phẩm"), "Danh mục" (dropdown showing "Điện tử"), "Đơn giá" (placeholder "VD: 6.500.000"), and "Số lượng" (placeholder "Nhập số lượng"). The buttons "Thêm sản phẩm" and "Quay lại danh sách" are also present.

Hình 3.8: Giao diện form hiển thị lỗi khi chưa điền đầy đủ thông tin

3.3.4 Giao diện responsive



Hình 3.9: Giao diện responsive của trang Home

Quản lý	Thời trang	420.000	18	Còn hàng	Sửa
jean nam					Xoá
Sữa tươi	Thực phẩm	32.000	50	Còn hàng	Sửa
Vinamilk 1L					Xoá
Mì gói	Thực phẩm	4.000	200	Còn hàng	Sửa
Hào Hào					Xoá
Nồi cơm điện	Gia dụng	890.000	12	Còn hàng	Sửa
Sharp					Xoá
Máy xay sinh tố	Gia dụng	650.000	7	Còn hàng	Sửa
					Xoá
ssdfg	Thực phẩm	54.454	42	Còn hàng	Sửa
					Xoá
rgr	Điện tử	3.546.546	4645	Còn hàng	Sửa
					Xoá
Trà sữa	Thực phẩm	20.000	20	Còn hàng	Sửa
					Xoá
Trà đào	Thực phẩm	15.000	20	Còn hàng	Sửa
					Xoá

Hình 3.10: Giao diện của bảng dữ liệu có scroll ngang khi ở responsive

CHƯƠNG 4. BÀI TẬP ỨNG DỤNG

4.1 Bài 1: Navbar

4.1.1 Yêu cầu

Tạo thanh điều hướng (Navbar) có ba mục: Home, List, Create. Mục đang được chọn (URL đang active) phải được highlight để người dùng nhận biết vị trí hiện tại trong ứng dụng.

4.1.2 Phân tích kỹ thuật

Bài toán bao gồm 2 yêu cầu chính: đảm bảo điều hướng không reload trang (react-router) và tự động highlight mục đang active. Cách giải quyết:

- **Điều hướng:** Dùng prop `as={NavLink}` để thay thế thẻ `<a>` mặc định bằng `NavLink` của React Router. Khi nhấn, ứng dụng chỉ render lại component tương ứng thay vì tải lại toàn bộ trang.
- **Auto highlight:** `NavLink` tự động gán class `.active` vào mục có prop `to` khớp với URL hiện tại. React-Bootstrap nhận diện class này và áp dụng style highlight tương ứng.
- **Responsive:** Prop `expand="lg"` trên Navbar cho phép thanh nav thu gọn thành nút hamburger trên màn hình nhỏ hơn 992px. Cần thêm `Navbar.Toggle` và `Navbar.Collapse` để chức năng này hoạt động

4.1.3 Code minh họa

```
<Navbar bg="primary" variant="dark" expand="lg" fixed="top">
  <Container fluid>
    <Navbar.Brand as={NavLink} to="/home">
      <i className="bi bi-box-fill"></i> Mini-app
    </Navbar.Brand>

    <Navbar.Toggle aria-controls="basic-navbar-nav" />
```

```
<Navbar.Collapse id="basic-navbar-nav">

  <Nav className="ms-auto">

    <Nav.Link as={NavLink} to="/">

      Home

    </Nav.Link>

    <Nav.Link as={NavLink} to="/list">

      Danh sách

    </Nav.Link>

    <Nav.Link as={NavLink} to="/add">

      Thêm sản phẩm mới

    </Nav.Link>

    <NavDropdown title="Bảo cáo" id="basic-nav-dropdown">

      <NavDropdown.Item onClick={handleDevelopingFeature}>

        Thống kê

      </NavDropdown.Item>

      <NavDropdown.Item onClick={handleDevelopingFeature}>

        Xuất file

      </NavDropdown.Item>

      <NavDropdown.Item onClick={handleDevelopingFeature}>

        In ấn

      </NavDropdown.Item>

    </NavDropdown>

  </Nav>

</Navbar.Collapse>
```

```
</Container>

</Navbar
```

4.1.4 Giải thích chi tiết

- `bg="primary" variant="dark"`: Đặt màu nền xanh và chữ trắng cho thanh điều hướng. Hai prop này cần khớp với nhau - `variant="dark"` yêu cầu nền tối để chữ trắng hiển thị đúng.
- `<Nav.Link as={NavLink} to="/">`: `as={NavLink}` → giúp điều hướng mà không cần reload lại trang; `to="/"` → điều hướng đến đúng URL (ví dụ: home, list,...).
- `ms-auto`: Đẩy toàn bộ menu về phía phải của Navbar, tạo bố cục chuẩn.

4.2 Bài 2: Table + Modal

4.2.1 Yêu cầu

Dùng table để hiển thị danh sách 10 bản ghi sản phẩm. Mỗi hàng có nút Xóa; khi nhấn Xóa sẽ mở Modal xác nhận. Nếu xác nhận thì xóa bản ghi khỏi danh sách, nếu huỷ thì đóng Modal và giữ nguyên dữ liệu.

4.2.2 Phân tích kỹ thuật

- Quản lý dữ liệu: Dùng `useState` để lưu mảng sản phẩm. Khi xóa, dùng `.filter()` tạo mảng mới không chứa phần tử đã chọn thay vì mutate trực tiếp (đây là nguyên tắc immutability của React).
- Liên kết nút Xóa với Modal: Khi nhấn nút "Xóa" tại một hàng, ứng dụng sẽ cập nhật state `deleteProduct` bằng dữ liệu của sản phẩm đó. Việc `deleteProduct` nhận giá trị mới (khác null) sẽ kích hoạt việc hiển thị Modal xác nhận.
- Để hiển thị danh sách 10 bản ghi, ta sử dụng Component `<Table>` để đảm bảo tính responsive. Bên cạnh đó, ta còn sử dụng phương thức `.map()` để

duyệt qua mảng `products`; với mỗi sản phẩm, React sẽ tự động tạo ra một hàng (`<tr>`) tương ứng.

4.2.3 Code minh họa

Code cho Table hiển thị danh sách 10 bản ghi:

```
<Table responsive hover className="align-middle">
  <thead className="bg-light">
    <tr className="text-secondary small text-uppercase fw-bold">
      <th className="py-3 border-0">STT</th>
      <th className="py-3 border-0">Tên sản phẩm</th>
      <th className="py-3 border-0">Danh mục</th>
      <th className="py-3 border-0">Đơn giá</th>
      <th className="py-3 border-0">Kho</th>
      <th className="py-3 border-0">Trạng thái</th>
      <th className="py-3 border-0">Thao tác</th>
    </tr>
  </thead>

  <tbody>
    {products.map((p, index) => {
      const statusInfo = getStatusDetail(p.qty);

      return (
        <tr key={p.id} className="border-bottom border-light">
          <td className="text-muted">{index + 1}</td>

          <td
            className="fw-bold text-dark py-3"
            style={{ width: "30%" }}
          >
```

```

        {p.name}
      </td>

      <td>

        <Badge

          bg="primary"

          className="bg-opacity-10 text-primary px-3 py-2
fw-normal"

        >

          {p.cat}

        </Badge>
      </td>

      <td className="fw-bold text-secondary">{p.price}</td>
      <td className="fw-bold">{p.qty}</td>

      <td className="text-center">

        <Badge

          bg={statusInfo.variant}

          className={`bg-opacity-10 text-${statusInfo.variant}
px-3 py-2 fw-normal`}

          style={{ minWidth: "85px" }}

        >

          {statusInfo.text}

        </Badge>
      </td>

      <td className="text-center">

        <Button

          variant="white"

```

```

        className="border shadow-sm me-2 btn-sm px-3"

        onClick={() => handleEdit(p) }

      >
        Sửa
      </Button>

      <Button

        variant="white"

        className="border shadow-sm btn-sm px-3"

        onClick={() => handleOpenDeleteModal(p) }

      >
        Xoá
      </Button>
    </td>
  </tr>
);
}}
</tbody>
</Table>

```

Code Modal xác nhận xóa:

```

const handleConfirmDelete = () => {
  setProducts(products.filter((product) => product.id !== deleteProduct.id));
  setDeleteProduct(null);
  showAlert("warning", "Đã xóa sản phẩm thành công!");
};

<Button

  variant="white"

```

```

        className="border shadow-sm btn-sm px-3"

        onClick={() => handleOpenDeleteModal(p)}

    >
        Xóa
    </Button>

```

4.2.4 Giải thích chi tiết

- `products.filter((product) => product.id !== deleteProduct.id)`: Tạo mảng mới gồm tất cả phần tử ngoại trừ phần tử có id trùng. Đây là cách xóa đúng trong React - không dùng `.splice()` vì `splice` mutate mảng gốc, React sẽ không detect được thay đổi.
- `const statusInfo = getStatusDetail(p.qty);`: Gọi lại hàm tính số lượng tồn kho để hiển thị trạng thái còn hay hết hàng cho người dùng biết.
- `onClick={() => handleOpenDeleteModal(p)}`: Khi nhấn nút Xóa, sự kiện `onClick` sẽ gửi dữ liệu sản phẩm vào hàm xử lý. Việc này giúp ứng dụng biết chính xác người dùng muốn xóa sản phẩm nào để hiển thị thông tin lên Modal xác nhận.
- Sử dụng thẻ `<Table>` để tạo bảng.

4.3 Bài 3: Form + Alert

4.3.1 Yêu cầu

Tạo một biểu mẫu thêm sản phẩm mới gồm ít nhất 3 trường (Tên, Giá, Số lượng). Thực hiện kiểm tra dữ liệu (validation): nếu để trống sẽ hiển thị Alert lỗi (danger); nếu điền đầy đủ sẽ hiển thị Alert thành công (success) và reset form.

4.3.2 Phân tích kỹ thuật

- **Cấu trúc Form:** Sử dụng `Form.Group` để bao bọc các nhãn và ô nhập liệu. Kết hợp `Row` và `Col` để dàn trang các trường "Giá" và "Số lượng" trên cùng một hàng (Grid layout).
- **Validation:** Sử dụng thuộc tính `validated` của `React-Bootstrap`. Khi submit, hàm xử lý sẽ kiểm tra `checkValidity()`.
- **Phản hồi người dùng (Alert):** Dựa vào kết quả validation, trạng thái của `Alert (variant)` sẽ thay đổi linh hoạt. Để tự động ẩn thông báo, có thể sử dụng `setTimeout` trong logic xử lý.

4.3.3 Code minh họa

```
<Alert variant="warning" className="mb-3">
    Tính năng đang phát triển!
</Alert>

setTimeout(() => {
    setAlert({
        show: false,
        variant: "success",
        message: "",
    });
}, 5000);

<AppAlert
    show={alert.show}
    variant={alert.variant}
    message={alert.message}
    onClose={() => setAlert({ ...alert, show: false })}
/>
```

4.3.4 Giải thích chi tiết

- Khi hàm được gọi (ví dụ sau khi Xóa thành công), state lập tức được cập nhật thành `show: true` → `Alert` hiện ra.

- `setTimeout`: Đây là ý tưởng cốt lõi để tạo tính năng auto-dismiss. Nó mang lại trải nghiệm chuyên nghiệp giống các framework lớn (như Toastify), người dùng không cần bấm tắt thủ công.
- Tái sử dụng `<AppAlert />`: Thay vì viết lại mã HTML của `Alert` ở nhiều nơi (khi xóa, khi sửa lỗi, khi sửa thành công), state `alert` sẽ đóng vai trò cấu hình động. Tham số `variant` thay đổi ("danger", "success", "warning") sẽ lập tức đổi màu của khung thông báo Bootstrap.
- `event.preventDefault()`: Chặn hành vi tải lại trang (reload) mặc định của trình duyệt khi submit form, giúp ứng dụng hoạt động chuẩn theo mô hình Single Page Application (SPA).

CHƯƠNG 5. HƯỚNG DẪN CHẠY PROJECT VÀ KẾT LUẬN

5.1 Hướng dẫn chạy

5.1.1 Yêu cầu môi trường

Trước khi chạy project, máy tính cần được cài đặt các công cụ sau:

Công cụ	Phiên bản tối thiểu	Ghi chú
Node.js	V18.0 trở lên	Tải tại nodejs.org ; kiểm tra bằng lệnh <code>node -v</code>
npm	V9.0 trở lên	Đi kèm khi cài Node.js ; kiểm tra bằng lệnh <code>npm -v</code>
git	Bất kỳ	Cần thiết để clone repo

Bảng 5.1: Bảng môi trường cần thiết để chạy project

5.1.2 Các bước thực hiện

Thực hiện lần lượt các bước sau để khởi động ứng dụng:

- **Bước 1:** Clone repo từ GitHub

```
git clone [GITHUB_REPO_URL]
```

```
cd [TEN_THU_MUC_PROJECT]
```

Thay [GITHUB_REPO_URL] bằng đường dẫn thực tế (https://github.com/dangngockimkhanh130605-svg/GKF23_Full-stack.git) và [TEN_THU_MUC_PROJECT] bằng tên thư mục project (tên thư mục project của nhóm là: mini-app-react).

- **Bước 2:** Cài đặt các thư viện phụ thuộc

```
npm install
```

Lệnh này đọc file package.json và tự động tải toàn bộ thư viện cần thiết (React, React-Bootstrap, React Router,...) vào thư mục node_modules. Quá trình này chỉ cần thực hiện một lần sau khi clone.

- **Bước 3:** Khởi động server phát triển

```
npm run dev
```

Vite sẽ khởi động development server. Sau khi thấy thông báo VITE v8.x.x ready in ... ms, mở trình duyệt và truy cập:

<http://localhost:5173/>

Cổng mặc định của Vite là 5173. Nếu cổng này đã bị chiếm, Vite tự động chọn cổng tiếp theo và hiển thị địa chỉ chính xác trong terminal.

5.1.3 Kiểm tra cài đặt thành công

Sau khi chạy npm run dev, kiểm tra các dấu hiệu sau để xác nhận project đã khởi động đúng:

- Terminal không hiện lỗi đỏ - chỉ hiển thị thông báo VITE ready và địa chỉ localhost
- Trình duyệt tải được trang chủ - thành Navbar hiển thị với bốn mục: Home, Danh sách, Thêm sản phẩm mới, Báo cáo
- Điều hướng hoạt động - nhấn vào từng mục menu, URL thay đổi và nội dung trang được cập nhật mà không cần reload
- Hot reload hoạt động - chỉnh sửa một file .jsx bất kỳ, trình duyệt tự cập nhật ngay lập tức mà không cần refresh

5.2 Kết luận

Trong quá trình thực hiện báo cáo giữa kỳ, nhóm đã nghiên cứu và ứng dụng thành công thư viện React-Bootstrap vào việc xây dựng một ứng dụng web quản lý sản phẩm hoàn chỉnh. Kết quả nhóm đạt được:

- **Về kiến thức:** Nhóm nắm được cách hoạt động của hệ sinh thái React-Bootstrap, hiểu rõ từng component, và biết cách kết hợp với React-State, React-Router trong một ứng dụng thực tế.
- **Về kỹ năng:** Thực hành được quy trình phát triển giao diện bằng component library - từ cài đặt, cấu hình, xử lý validation form, quản lý state đa component, đến tổ chức cấu trúc project hợp lý.
- **Về sản phẩm:** Mini-app hoàn chỉnh với đầy đủ chức năng CRUD, giao diện nhất quán và responsive trên cả desktop lẫn thiết bị di động.

TÀI LIỆU THAM KHẢO

- Accordion*. (n.d.). React Bootstrap. Retrieved May 3, 2026, from <https://react-bootstrap.netlify.app/docs/components/accordion>
- Buttons*. (n.d.). React Bootstrap. Retrieved May 3, 2026, from <https://react-bootstrap.netlify.app/docs/components/buttons>
- Cards*. (n.d.). React Bootstrap. Retrieved May 3, 2026, from <https://react-bootstrap.netlify.app/docs/components/cards>
- Install React Router*. (n.d.). W3Schools. Retrieved May 3, 2026, from https://www.w3schools.com/react/react_router.asp
- Navbars*. (n.d.). React Bootstrap. Retrieved May 3, 2026, from <https://react-bootstrap.netlify.app/docs/components/navbar>
- React Bootstrap Tutorial* [Film]. (n.d.). <https://www.youtube.com/watch?v=8pKjULHzs0s&t=691s>
- Tables*. (n.d.). React Bootstrap. Retrieved May 3, 2026, from <https://react-bootstrap.netlify.app/docs/components/table>